

Manejo de transacciones SQL

Ing. Ricardo Tillerio

¿Qué son?

TRANSACCIONES SQL

“Las transacciones son unidades recuperables de tareas de acceso a datos para la manipulación del contenido de las bases de datos, proporcionando las bases para la gestión de la concurrencia en entornos multiusuario”

TRANSACCIONES SQL

- Una transacción es una unidad de la ejecución de un programa que accede y posiblemente actualiza varios elementos de datos.
- Una transacción es un conjunto de instrucciones SQL, generalmente de duración breve, que accede y actualiza una parte también generalmente pequeña de la base de datos, pudiendo controlar múltiples transacciones que se ejecutan en paralelo sobre una misma base de datos que corre en un sistema que puede fallar.
- Una transacción es una secuencia de instrucciones (sentencias SQL) relacionadas, que deben ser tratadas como una unidad indivisible.
- **Una transacción es atómica;** es decir, se hace todo el trabajo o no se hace nada.

TRANSACCIONES SQL

- El Procesamiento de Transacciones tiene como objetivo las cuestiones referentes a la integridad de la base de datos, recuperación y control de la concurrencia.
- Los objetivos del Gestor de Transacciones del SMDB son:
 - Evitar que las transacciones interfieran unas con otras al ejecutar en paralelo,
 - Garantizar que la base de datos no sea dañada en forma irreparable por caídas, ya sea del sistema en sí o de alguna de las transacciones.

- Por ejemplo, si realizamos una transferencia para pagar una compra realizada vía Internet, el proceso estaría formado por un conjunto de operaciones que deberán ser realizadas para que el proceso de pago tenga éxito:
 1. Comprobación de existencia, validez y operatividad de cuenta.
 2. Comprobación de si hay saldo en cuenta.
 3. Comprobación de datos de la cuenta destino.
 4. Retiro de dinero de nuestra cuenta.
 5. Abono de dinero en la cuenta de destino.
- Las 5 operaciones deben tener éxito o fallar todas en conjunto.
- Las transacciones se relacionan con la gestión de la concurrencia y los bloqueos. En el proceso de pago los 3 primeros pasos realmente no implican modificación alguna de datos, por lo que si fallan no dejarían la base de datos en un estado inconsistente, sin embargo ¿Qué pasaría si al mismo tiempo que se está realizando nuestra transferencia, entra en nuestra cuenta un cargo diferido que teníamos pendiente? Sería posible que de repente nos quedásemos sin saldo para realizar la operación actual.

TRANSACCIONES SQL

- Una transacción es una serie de operaciones que se ejecutan entre las sentencias **BEGIN TRANSACTION** (comienzo de transacción) y **END TRANSACTION** (fin de transacción), desde una aplicación o desde consola SQL.

```
START TRANSACTION;
```

```
    <sentencia>;
```

```
    <sentencia>;
```

```
COMMIT;
```

TRANSACCIONES SQL

- Consideremos la siguiente tabla de cuentas de banco:

```
CREATE TABLE Accounts (  
    acctId INTEGER NOT NULL PRIMARY KEY,  
    balance DECIMAL(11,2) CHECK (balance >= 0.00)  
);
```

- Transacción SQL de transferencia de dinero entre cuentas bancarias (por ejemplo 100 \$)

```
BEGIN TRANSACTION;  
    UPDATE Accounts SET balance = balance - 100 WHERE acctId = 101;  
    UPDATE Accounts SET balance = balance + 100 WHERE acctId = 202;  
COMMIT;
```

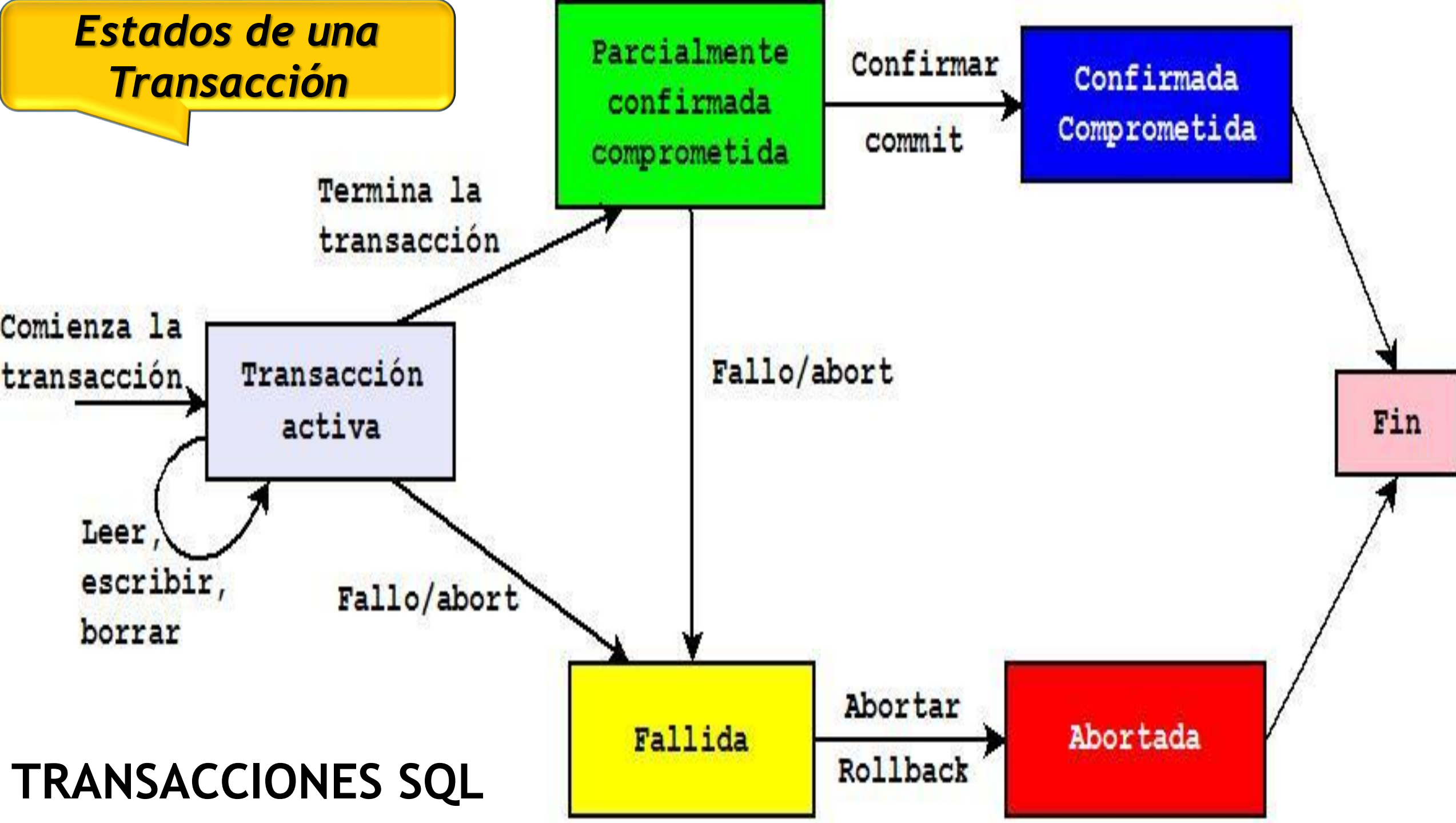
TRANSACCIONES SQL

Todo SMBD maneja los siguientes conceptos sobre las transacciones:

- Transacciones son un conjunto de acciones que altera el estado original de los datos y forman una sola unidad.
- Todo lenguaje que manipula los datos (DML) es administrado por las transacciones.
- Las transacciones pueden interrumpir un conjunto de acciones o hacerlas permanentes.
- Mantiene la integridad de los datos cuando alguna acción falla.

- Comandos básicos para el control de las transacciones MySQL:
 1. **BEGIN:** inicia la transacción.
 2. **COMMIT:** guardar los cambios en la base de datos.
 3. **ROLLBACK:** deshacer los cambios que se hubieran hecho en la transacción.
 4. **SAVEPOINT:** crea checkpoints o puntos de chequeo en la transacción, para poder deshacer la transacción hasta esos puntos.
 5. **ROLLBACK TO:** deshacer los cambios que se hubieran hecho en la transacción hasta un checkpoint específico.
 6. **RELEASE SAVEPOINT:** libera un checkpoint o punto de chequeo específico en la transacción.
- Los comandos de control de transacciones se usan sólo con SELECT, INSERT, DELETE y UPDATE. No pueden utilizarse creando tablas o vaciándolas porque las operaciones se guardan automáticamente en la base de datos.

Estados de una Transacción



TRANSACCIONES SQL

TRANSACCIONES SQL

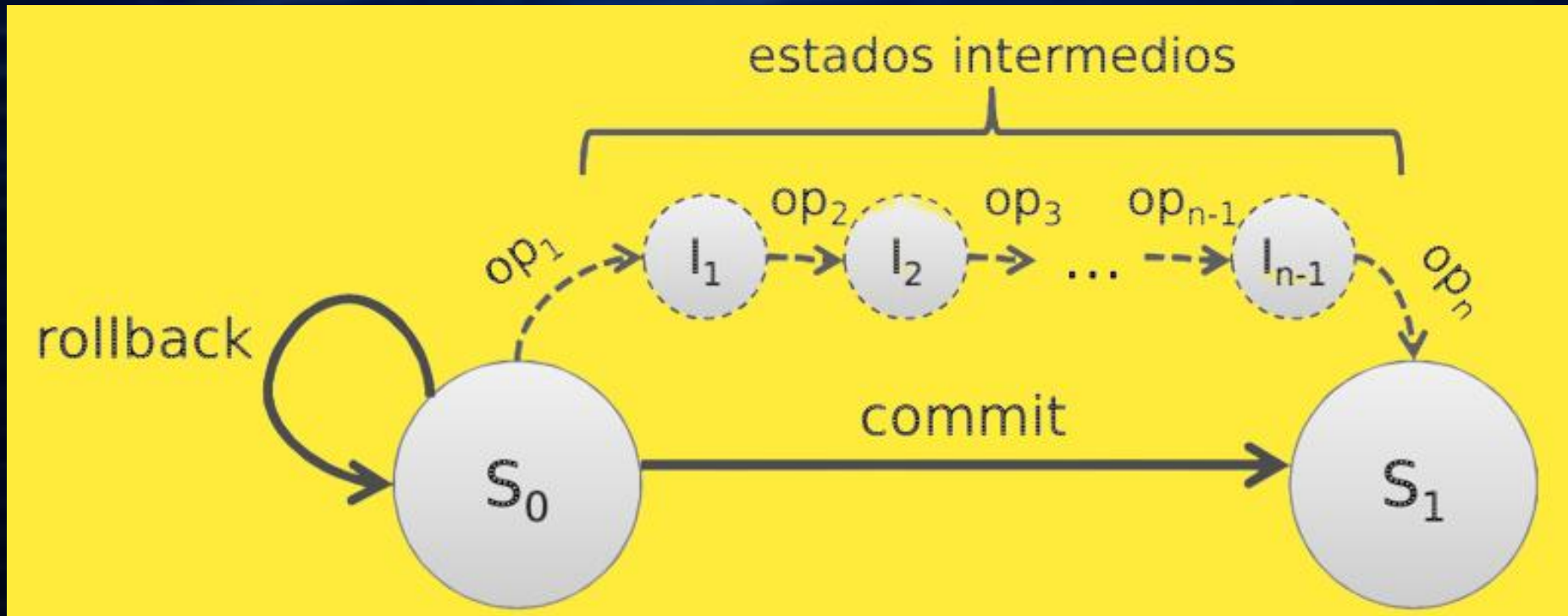
- **Transacción activa:** el estado inicial; la transacción permanece en este estado durante su ejecución.
- **Parcialmente confirmada/comprometida:** después de ejecutarse la última instrucción.
- **Fallida:** tras descubrir que no puede continuar la ejecución normal.
- **Abortada:** después del retroceso/rollback de la transacción y de haber restablecido la base de datos al estado que tenía antes del comienzo de la transacción.
- **Confirmada/comprometida:** tras completarse con éxito.

La gestión de transacciones implica algunas características importantes llamadas propiedades ACID (Atomicity, Consistency, Isolation and Durability), estas propiedades ACID, garantizan que la base de datos mantenga un estado correcto a pesar de una falla de concurrencia o del sistema, durante las transacciones:

- **Atomicidad:** asegura que todas las operaciones dentro de la secuencia de trabajo se completen satisfactoriamente. Si no es así, la transacción se abandona en el punto del error y las operaciones previas retroceden a su estado inicial.
- **Consistencia:** asegura que la base de datos cambie estados en una transacción exitosa.
- **Aislamiento:** permite que las operaciones sean aisladas y transparentes unas de otras.
- **Durabilidad:** asegura que el resultado o efecto de una transacción completada se haga permanente, así ocurra un error del sistema.

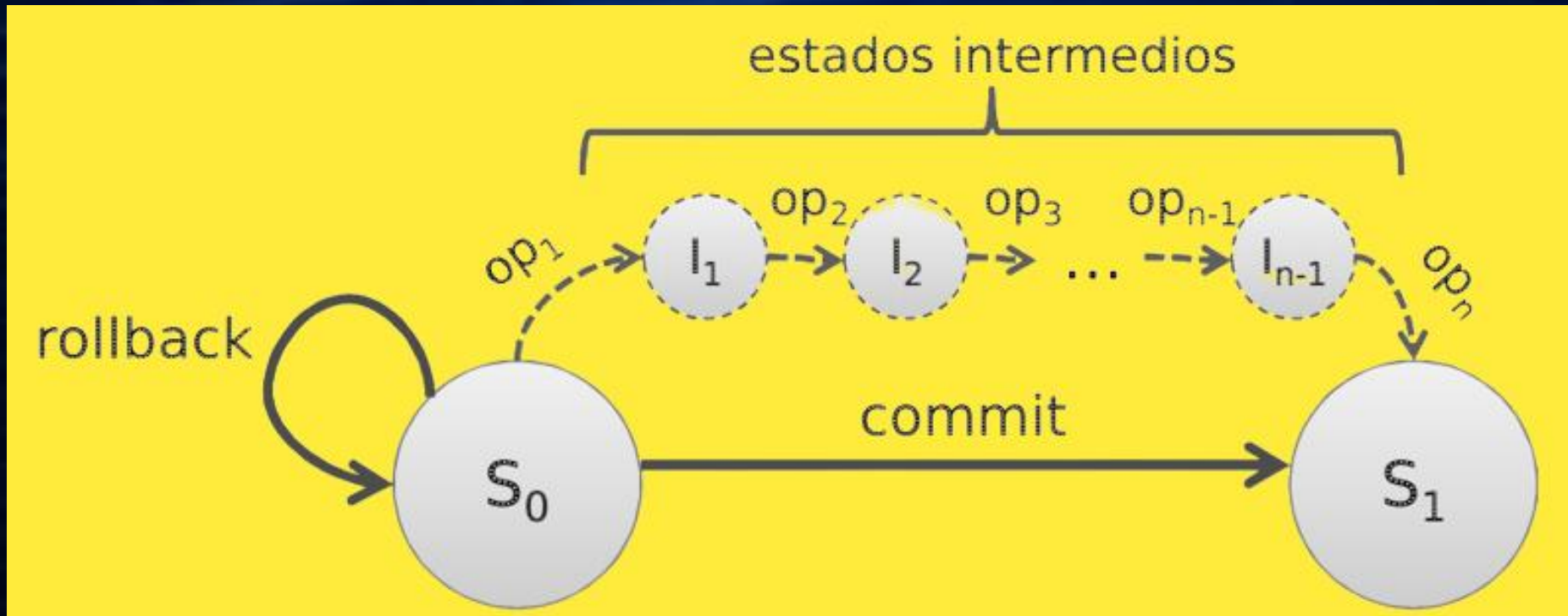
Atomicidad:

- Si se confirma (commit), se ejecutan todas las operaciones; si se cancela (rollback), no se ejecuta ninguna.
- No se puede ejecutar parcialmente, dejando la base de datos en un estado intermedio (li).



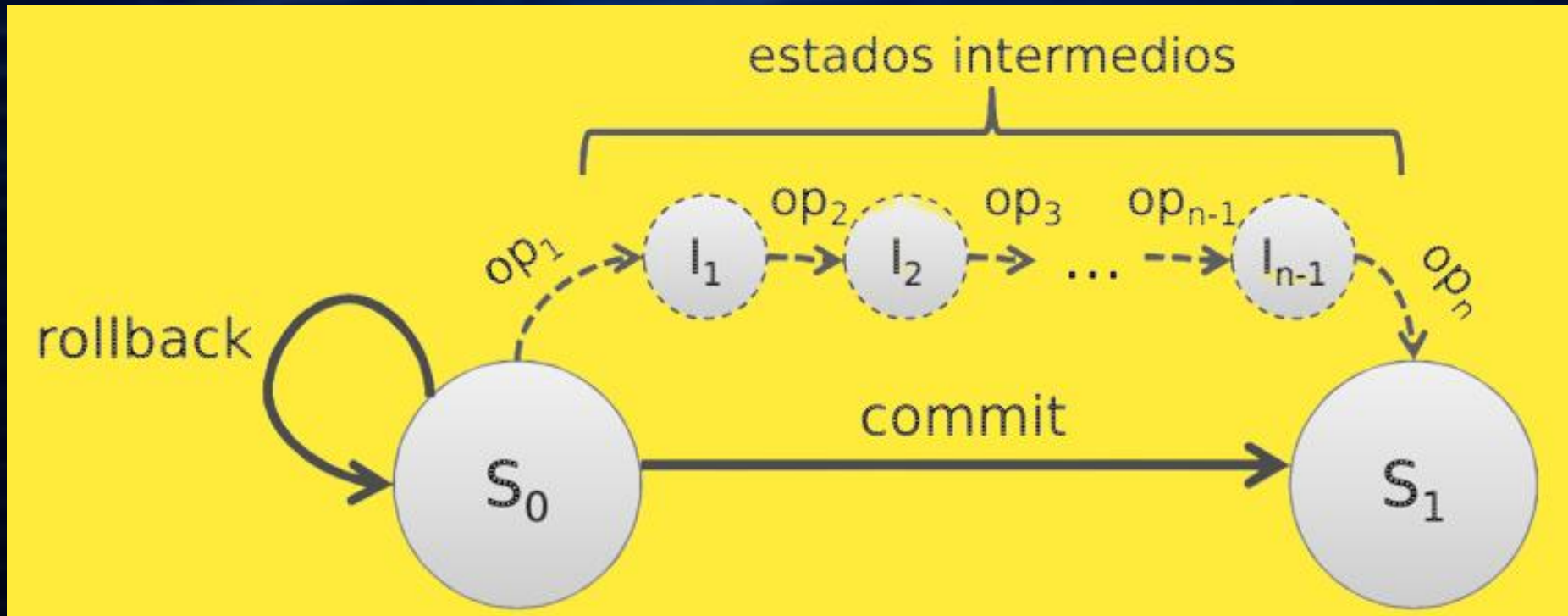
Consistencia:

- A partir de un estado inicial consistente (S_0), deja la base de datos en otro estado consistente (S_1), que respeta todas las reglas de integridad.
- Los estados intermedios ($I_1..n$) no son necesariamente consistentes.



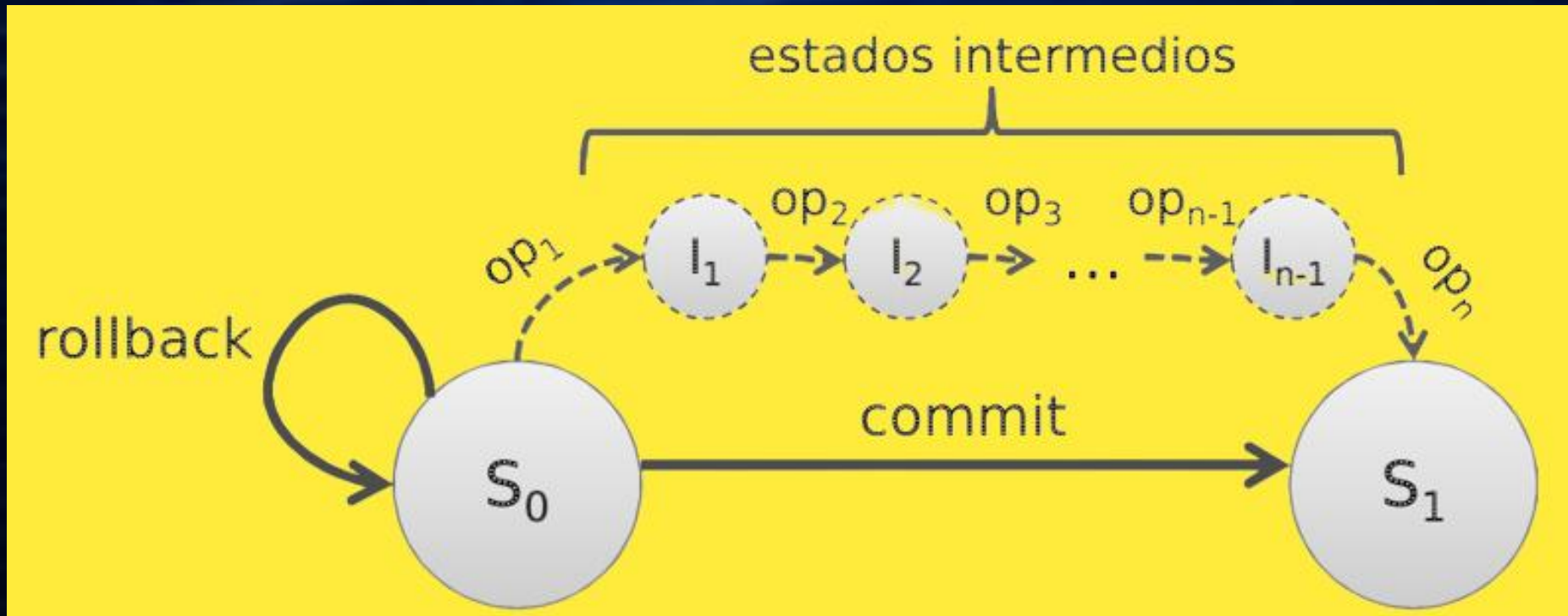
Aislamiento:

- Se realiza como si fuera la única transacción ejecutándose en la base de datos.
- Los estados intermedios ($I_1..n$) no deben ser visibles para otras transacciones. Es la propiedad menos desarrollada en los SGBD.



Durabilidad:

- Si se confirma (commit), los cambios en la base de datos se hacen permanentes.
- El SGBD es responsable de que los cambios sean permanentes, incluso aunque se produzca un **fallo**.



TRANSACCIONES SQL

Por regla general en los SMBD relacionales modernos disponen de tres tipos de transacciones según la forma de iniciarlas:

- **De confirmación automática (auto commit):** el SMBD inicia una transacción automáticamente por cada operación que actualice datos. De este modo mantiene siempre la consistencia de la base de datos, aunque puede generar bloqueos.
- **Implícitas:** cuando el SMBD comienza una transacción automáticamente cada vez que se produce una actualización de datos, pero el que dicha transacción se confirme o se deshaga, lo debe indicar el programador.
- **Explícitas:** son las que iniciamos nosotros “a mano” mediante instrucciones SQL, somos nosotros, los programadores, los que indicamos qué operaciones va a abarcar. Una transacción explícita se define de manera general con una instrucción que marca su inicio, y dos posibles instrucciones que marcan su final en función de si debe tener éxito o debe fracasar en bloque.

TRANSACCIONES SQL

- Una transacción es vista por el SGBD como una serie, o lista, de acciones. Las acciones que pueden ejecutarse en una transacción incluyen lecturas y escrituras de objetos de base de datos.
- Usaremos pseudocódigo para describir las transacciones, y denotaremos las acciones como: L(O), E(O), read O, write O. Además de leer y escribir, cada transacción se debe especificar como última acción bien sea Comprometer (COMMIT) o Abortar (ROLLBACK).
- Hay dos suposiciones importantes para la planificación de transacciones:
 1. Las transacciones interactúan mutuamente sólo a través de operaciones de lectura y escritura de la base de datos; por ejemplo, no se permite el intercambio de mensajes.
 2. Una base de datos es una colección fija de objetos independientes. Cuando se añaden o eliminan objetos de una base de datos o existen relaciones entre objetos de la base de datos que se quieren aprovechar por rendimiento, surgen problemas adicionales.
- Si se viola la primera suposición, el SGBD no tiene forma de detectar o prevenir inconsistencias causadas por estas interacciones externas entre transacciones, y es responsabilidad del programador de la aplicación asegurar que el programa se comporte correctamente. La segunda suposición se tratará con el “Control de Concurrency”.

TRANSACCIONES SQL

- Una planificación es una lista de acciones (lectura, escritura, abortar o comprometer) de un conjunto de transacciones, y el orden en el que dos acciones de una transacción T aparecen en una planificación debe ser el mismo que el orden en el que aparecen en T .
- Intuitivamente, una planificación representa una secuencia de ejecución real o potencial. Por ejemplo, la planificación de la figura muestra un orden de ejecución de las acciones de dos transacciones $T1$ y $T2$. Se avanza en el tiempo de arriba hacia abajo de una fila a la otra.
- Una planificación describe solo las acciones de base de datos. Pero en una transacción se pueden realizar otras acciones, tales como leer o escribir en archivos del sistema operativo, evaluar expresiones aritméticas, entre otras; sin embargo, se supone que estas acciones no afectan a otras transacciones, por lo que no se describen en las planificaciones, mas si en la programación.

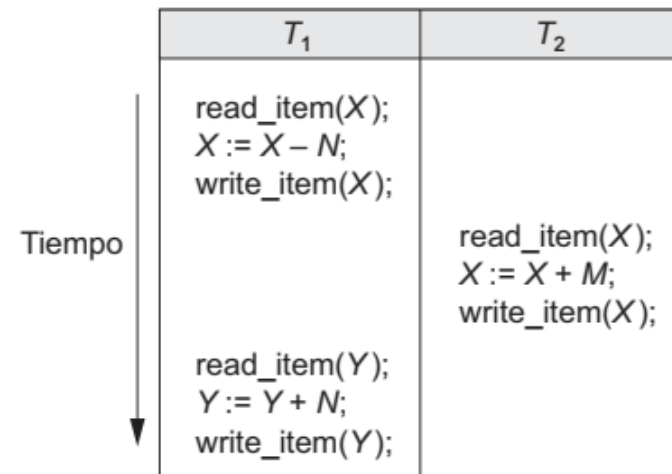
$T1$	$T2$
$L(A)$	
$E(A)$	
	$L(B)$
	$E(B)$
$L(C)$	
$E(C)$	



- Obsérvese que la planificación de la figura anterior no contiene ninguna acción de abortar o comprometer para ninguna de las transacciones. Una planificación que contenga una de esas acciones para cada transacción se denomina una “**planificación completa**”.
- Si las acciones de las diferentes transacciones no están entrelazadas (es decir, las transacciones se ejecutan desde el principio hasta el final, una por una), la planificación se denomina “**planificación secuencial, serial, o serie**”.

Hora	S	T
t1	BEGIN TRANS	
t2	read x	
t3	$x = x * 3$	
t4	write x	
t5	read y	
t6	$y = y + 2$	
t7	write y	
t8	COMMIT	
t9		BEGIN TRANS
t10		read x
t11		$x = x + 2$
t12		write x
t13		read y
t14		$y = y + 5$
t15		write y
t16		COMMIT

Planificación completa serial



Planificación no completa no serial

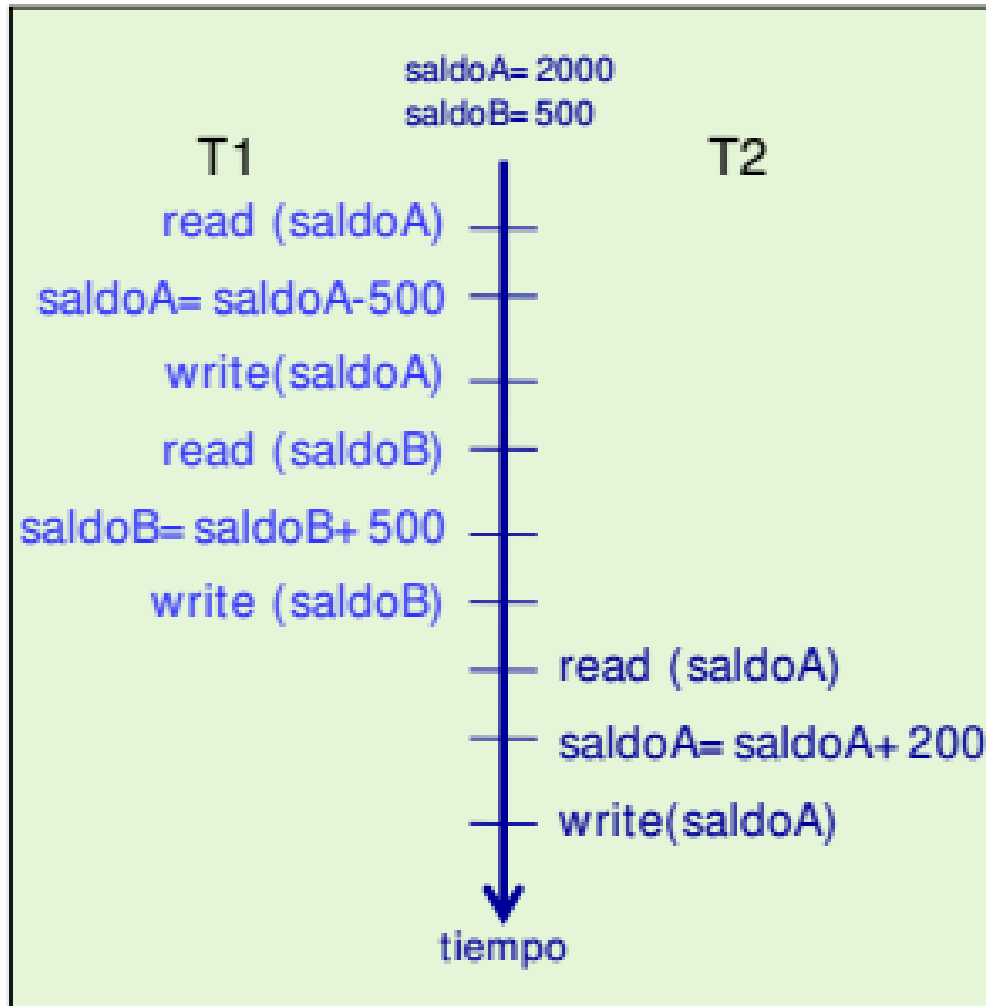
Planificación serial de transacciones:

- Significa que las transacciones se ejecutan una después de otra, sin cualquier intercalación de operaciones. En una ejecución serial no hay interferencia entre las transacciones, puesto que en cualquier momento dado sólo una se está ejecutando.
- Al tener ejecuciones seriales, son importantes los factores siguientes:
 - Si dos transacciones únicamente leen los ítem de datos, ellas no entran en conflicto y el orden no es importante .
 - Si dos transacciones operan (ya sea lectura o escritura) sobre los ítem de datos separados completamente, no hay conflicto y no es importante el orden .
 - Si una transacción escribe sobre un ítem de datos y otro lee o escribe sobre este mismo ítem, entonces el orden de ejecución sí es importante.
- **Operaciones conflictivas:** dos operaciones son conflictivas si cumplen con todos los enunciados siguientes:
 - Pertenecen a transacciones diferentes.
 - Acceden al mismo ítem de datos.
 - Al menos una escribe en el mismo ítem.

Ejemplos de planificaciones de transacciones:

Plan serie o sucesión

(T1 seguida de T2 o bien T2 seguida de T1)



Plan no serie

(cualquier combinación que intercale acciones de T1 y T2)

